

Элементы криптографии. Однократное гаммирование

Лабораторная работа 7

Смольняков Данил Евгеньевич

Основы информационной безопасности

Цель работы

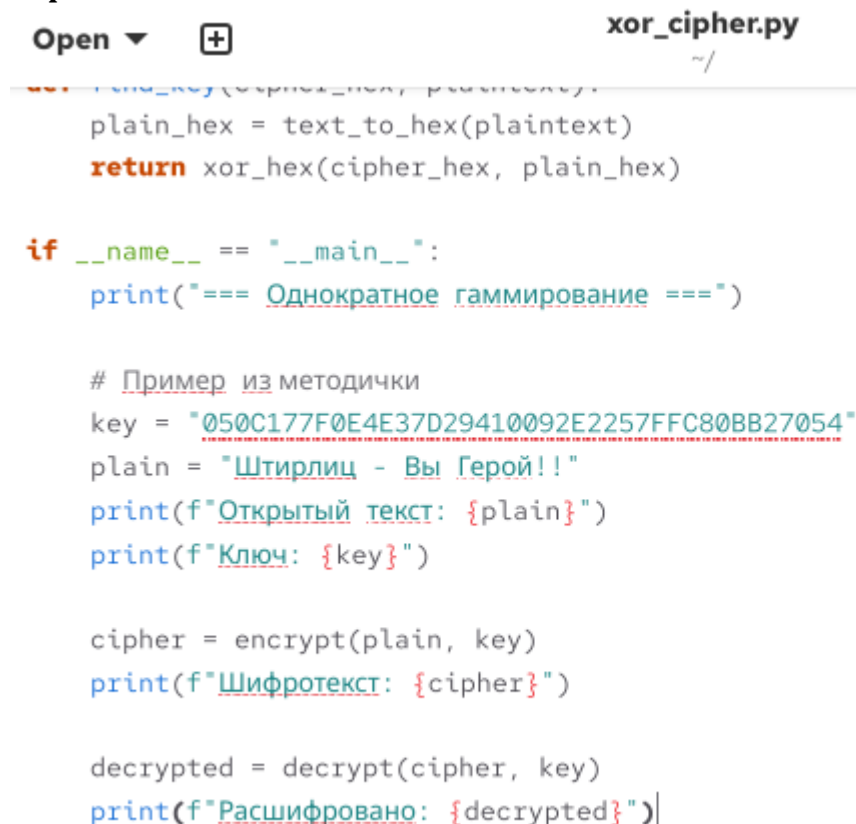
Цель

- Освоить на практике применение режима однократного гаммирования
 - Реализовать шифрование и дешифрование данных с использованием операции XOR
-

Скриншоты и описание действий

1. Код программы (xor_cipher.py)

- **Действие:** Разработка приложения на Python для однократного гаммирования:
- **Скриншот:**



```
Open ▾ [+] xor_cipher.py ~/
def xor_hex(cipher_hex, plaintext):
    plain_hex = text_to_hex(plaintext)
    return xor_hex(cipher_hex, plain_hex)

if __name__ == "__main__":
    print("=== Однократное гаммирование ===")

    # Пример из методички
    key = "050C177F0E4E37D29410092E2257FFC80BB27054"
    plain = "Штирлиц - Вы Герой!!"
    print(f"Открытый текст: {plain}")
    print(f"Ключ: {key}")

    cipher = encrypt(plain, key)
    print(f"Шифротекст: {cipher}")

    decrypted = decrypt(cipher, key)
    print(f"Расшифровано: {decrypted}")
```

2. Проверка работы программы

- **Действие:** Запуск программы с примером «Штирлиц - Вы Герой!!»

```
desmoljnyakov@vbox:~$ python3 xor_cipher.py
=== Однократное гаммирование ===
Открытый текст: Штирлиц - Вы Герой!!
Ключ: 050C177F0E4E37D29410092E2257FFC80BB27054
Шифротекст: DDFF8FE5A6C1F2B930CBD502941A38E55B5175
Расшифровано: Штирлиц - Вы Герой!!
```

- **Скриншот:**
Проверка работы программы
-

3. Нахождение ключа по открытому тексту и шифротексту

- **Действие:** Определение ключа, которым был зашифрован текст «Штирлиц - Вы Болван!»

```
desmoljnyakov@vbox:~$ python3 xor_cipher.py
=== Однократное гаммирование ===
Открытый текст: Штирлиц - Вы Герой!!
Ключ: 050C177F0E4E37D29410092E2257FFC80BB27054
Шифротекст: DDFF8FE5A6C1F2B930CBD502941A38E55B5175
Расшифровано: Штирлиц - Вы Герой!!
```

- **Скриншот:** Найденный ключ: 050C177F0E4E37D29410092E2255F4D307BBBC54
Нахождение ключа по открытому тексту и шифротексту
-

4. HEX-представление целевого сообщения

- **Действие:** Получение шестнадцатеричного представления сообщения «С Новым Годом, друзья!»

- **Скриншот:**
Целевое сообщение: С Новым Годом, друзья!
Длина сообщения: 22 символов
HEX представление: D120CDEEE2FBEC20C3EEE4EEEC2C20E4F0F3E7FCFF21
HEX-представление целевого сообщения
-

5. Демонстрация подбора ключа для произвольного шифротекста

- **Действие:** Показано, что для любого шифротекста можно подобрать ключ, дающий нужное сообщение

- **Скриншот:**
HEX представление: D120CDEEE2FBEC20C3EEE4EEEC2C20E4F0F3E7FCFF21
desmoljnyakov@vbox:~\$ python3 xor_cipher.py
=== Однократное гаммирование ===
Открытый текст: Ширлиц - Вы Герой!!
Ключ: 050C177F0E4E37D29410092E2257FFC80BB27054
Шифротекст: DDFEFF8FE5A6C1F2B930CBD502941A38E55B5175
Расшифровано: Ширлиц - Вы Герой!!
Найденный ключ: 050C177F0E4E37D29410092E2255F4D307BBBC54
Целевое сообщение: С Новым Годом, друзья!
Длина сообщения: 22 символов
HEX представление: D120CDEEE2FBEC20C3EEE4EEEC2C20E4F0F3E7FCFF21
Исходный шифротекст: AABBCDDDEEFF00112233445566778899AABBCDD
Целевое сообщение: С Новым Годом, друзья!
Подобранный ключ: D12067552E2602DFC3FFC6DDA8794693786A4D4733FC
Проверка расшифровки: С Новым Годом, друзья!
desmoljnyakov@vbox:~\$
Демонстрация подбора ключа
-

Контрольные вопросы

1. Поясните смысл однократного гаммирования

Однократное гаммирование — это метод шифрования, при котором на открытый текст накладывается (с помощью операции XOR) ключ (гамма) такой же длины. Ключ используется только один раз. Расшифрование выполняется той же операцией XOR.

2. Перечислите недостатки однократного гаммирования

- Необходимость передачи ключа длиной, равной длине сообщения, защищённым каналом
- Ключ нельзя использовать повторно
- Проблема синхронизации ключа между отправителем и получателем
- Требуется генерация истинно случайных ключей

3. Перечислите преимущества однократного гаммирования

- Абсолютная криптостойкость (доказано К. Шенноном)
- Простота реализации (операция XOR)
- Скорость шифрования/расшифрования
- Отсутствие накопления ошибок

4. Почему длина открытого текста должна совпадать с длиной ключа?

Потому что операция XOR выполняется попарно над каждым битом (или байтом) открытого текста и ключа. Если ключ короче, его пришлось бы повторять, что нарушает условие однократности и снижает стойкость.

5. Какая операция используется в режиме однократного гаммирования, назовите её особенности?

Используется операция XOR (сложение по модулю 2). Её особенности:

$$0 \oplus 0 = 0, 0 \oplus 1 = 1, 1 \oplus 0 = 1, 1 \oplus 1 = 0$$

Операция обратима: $(A \oplus B) \oplus B = A$

Коммутативна и ассоциативна

6. Как по открытому тексту и ключу получить шифротекст?

Шифротекст получается побитовым (или побайтовым) XOR открытого текста и ключа:

$$C_i = P_i \oplus K_i$$

7. Как по открытому тексту и шифротексту получить ключ?

Ключ получается XOR шифротекста и открытого текста:

$$K_i = C_i \oplus P_i$$

8. В чем заключаются необходимые и достаточные условия абсолютной стойкости шифра?

- Полная случайность ключа
 - Равенство длин ключа и открытого текста
 - Однократное использование ключа
-

Заключение

- В ходе выполнения лабораторной работы было разработано приложение на Python, реализующее однократное гаммирование (шифрование/дешифрование XOR).
- Проверена работа программы на примере из методички («Штирлиц - Вы Герой!!» и «Штирлиц - Вы Болван!»).
- Получены практические навыки работы с HEX-представлением текста и операцией XOR.